

Documentação Técnica — Cur10usX

Plataforma de Gestão Escolar Multi-Tenant Versão: 1.1.0 | Stack: Next.js 16 + React 19 + Prisma + PostgreSQL

Índice

1. [Visão Geral](#)
 2. [Stack Tecnológica](#)
 3. [Arquitetura do Sistema](#)
 4. [Frontend](#)
 5. [Backend \(API\)](#)
 6. [Base de Dados](#)
 7. [WebSocket](#)
 8. [Autenticação e Autorização](#)
 9. [DevOps](#)
 10. [Segurança](#)
 11. [Escalabilidade](#)
 12. [Guia para Desenvolvedores](#)
-

1. Visão Geral

Propósito

Cur10usX é uma plataforma moderna de gestão escolar desenhada para o contexto educacional angolano. Substitui processos manuais e sistemas dispersos por uma única plataforma unificada que cobre administração escolar, gestão académica, comunicação institucional e avaliação de desempenho.

Problema que Resolve

- Escolas angolanas utilizam processos manuais (papel, Excel) para gerir alunos, professores, notas, presenças e horários
- Inexistência de uma plataforma centralizada que respeite o currículo e ciclos de ensino angolanos
- Dificuldade de comunicação entre direção, professores, alunos e encarregados de educação
- Falta de portabilidade do histórico académico entre escolas

Visão da Plataforma

- **Multi-tenant:** Cada escola é uma organização independente com os seus próprios dados e branding
- **Role-based:** 5 perfis com permissões granulares (super_admin, school_admin, teacher, student, parent)
- **Real-time:** Notificações e atualizações em tempo real via WebSocket
- **Multi-language:** Suporte a PT, EN, FR, ES
- **Cloud-native:** Deploy em containers Docker, orquestração Kubernetes, database PostgreSQL na Neon

Funcionalidades Principais

Área	Funcionalidades
Autenticação	Email/password + Google OAuth, 2FA (TOTP), verificação de email, reset de password
Gestão Escolar	Multi-tenant com branding personalizado, ciclos de ensino angolanos
Gestão Académica	Turmas, disciplinas, cursos, matrículas, horários, exames, trabalhos
Avaliação	Motor de avaliação com trimestres, pesos configuráveis, fórmula de média, recurso
Presenças	Registo diário por aula com status (presente/ausente/atrasado)
Comunicação	Mensagens internas, anúncios por prioridade, tickets de suporte
Notificações	Em tempo real via WebSocket + notificações na plataforma
Importação/Exportação	Bulk import CSV/XLSX, exportação de dados
Relatórios	Dashboard analítico com gráficos e estatísticas
Conformidade	GDPR (exportação e eliminação de dados), termos legais

2. Stack Tecnológica

Camada	Tecnologia	Versão	Razão da Escolha
Framework	Next.js	16.1.6	Full-stack unificado (frontend + API routes), App Router, React Server Components
UI	React	19.2.3	Ecosistema maduro, Server Components, concurrent features
Linguagem	TypeScript	5.9.3	Type safety, melhor DX, prevenção de erros em runtime
Estilização	Tailwind CSS	4	Utility-first, build-time generation, sem runtime CSS-in-JS
ORM	Prisma	6.19.2	Type-safe queries, migrations automáticas, schema declarativo, suporte a PostgreSQL
BD	PostgreSQL (Neon)	-	Robustez relacional, JSON fields para configs flexíveis, performance em queries complexas
Auth	Auth.js (NextAuth v5)	5.0.0-beta.30	Suporte nativo a Next.js, múltiplos providers, JWT + database sessions
Validação	Zod	4.3.6	Type-safe schemas, integração com TypeScript, composição de schemas
Estado Global	Redux Toolkit	2.11.2	Estado previsível, middleware, devtools, suporte a async thunks
Charts	Recharts	3.7.0	Declarativo, React-native, leve, boa personalização
WebSocket	ws	8.20.0	Leve, performante, sem dependências pesadas, servidor standalone
2FA	Speakeasy	2.0.0	TOTP standard, integração com Google Authenticator
QR Code	qrcode	1.5.6	Geração de QR codes para setup de 2FA
Email	Resend	6.9.3	API moderna, entregabilidade alta, suporte a React Email
Upload	Vercel Blob	2.3.3	Edge uploads, CDN integrada, sem necessidade de servidor de ficheiros
PDF	jsPDF	4.2.1	Geração client-side de relatórios e certificados
Contentorização	Docker	-	Consistência entre ambientes, isolamento, CI/CD
Orquestração	Kubernetes	-	Escalabilidade horizontal, self-healing, rolling updates

Trade-offs e Decisões Técnicas

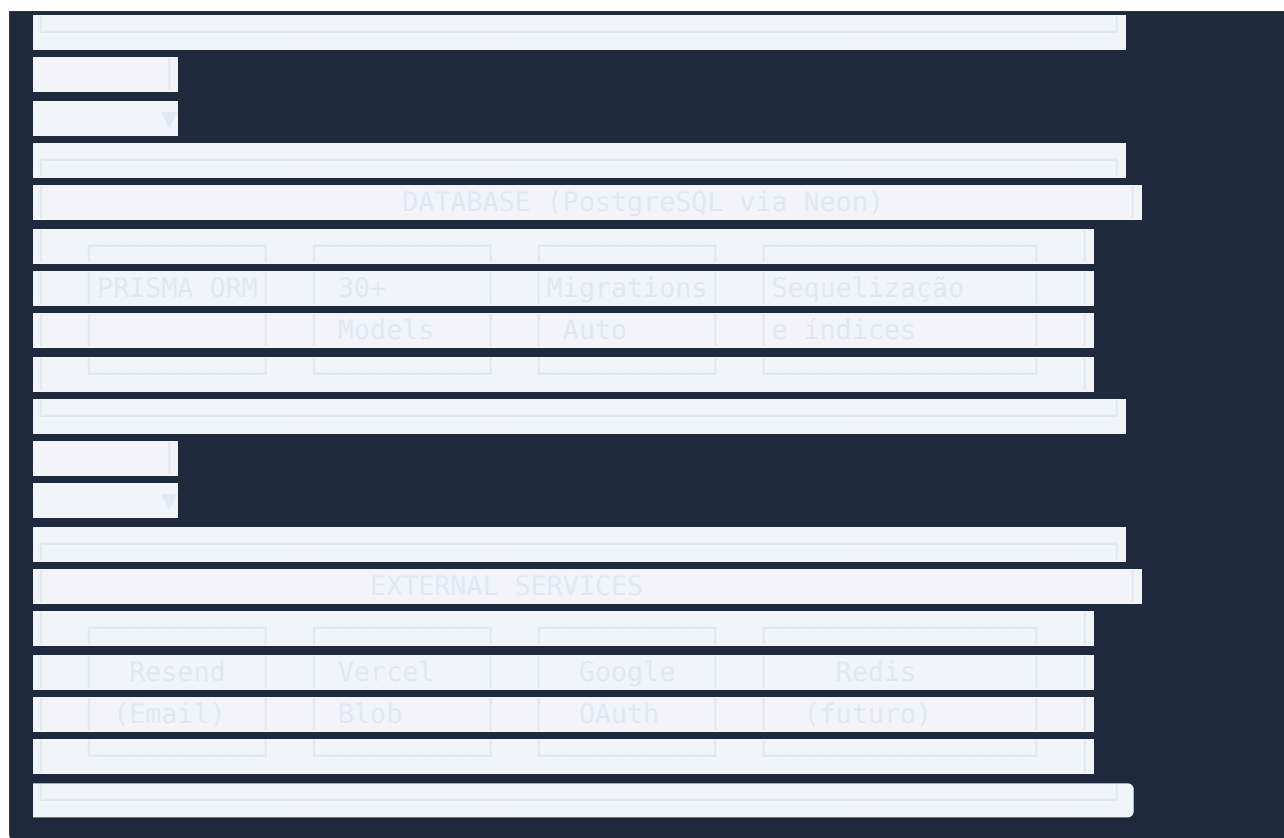
Decisão	Alternativa	Razão
Next.js API Routes vs NestJS	NestJS separado	Escolhemos API Routes pela simplicidade de deploy (monolito), partilha de tipos com frontend, e redução de overhead operacional. Para escalabilidade futura, extrair para microserviços é possível.

Decisão	Alternativa	Razão
JWT Sessions vs Database Sessions	Database sessions	JWT permite stateless auth, escala horizontalmente sem shared session store. Trade-off: tokens não revogáveis (mitigado por sessionVersion).
Tailwind vs Styled Components	Styled Components	Build-time CSS (zero runtime), bundle menor, consistência via utility classes. Trade-off: JSX mais verboso.
Prisma vs Drizzle	Drizzle ORM	Prisma tem melhor DX (schema declarativo, migrations automáticas, studio). Trade-off: performance marginalmente inferior em queries complexas.
Monolito vs Microserviços	Microserviços	Monolito simplifica deploy, DevOps e desenvolvimento inicial. Para escalar, o domínio está bem delimitado para futura extração.

3. Arquitetura do Sistema

Arquitetura Geral — Monolito Modular





Estrutura de Pastas (Frontend + Backend)

```
curl0us/  
├── prisma/  
│   ├── schema.prisma      # Schema da base de dados (952 linhas, 30+ models)  
│   ├── seed.ts            # Dados de seed para desenvolvimento  
│   ├── migrations/        # Migrações automáticas do Prisma  
│   └── data-backfill.sql   # Scripts de backfill manual  
├──  
├── src/  
│   ├── middleware.ts      # Edge middleware – proteção de rotas  
│   ├──  
│   ├── app/              # Next.js App Router  
│   │   ├── layout.tsx     # Root layout (providers, fonts)  
│   │   ├── page.tsx       # Landing page (público)  
│   │   ├──  
│   │   ├── (public)/      # Rotas públicas  
│   │   │   ├── aplicacao/  # Status de candidatura  
│   │   │   ├── termos/    # Termos de serviço  
│   │   │   └── privacidade/ # Política de privacidade  
│   │   ├──  
│   │   ├── (auth)/        # Rotas de autenticação  
│   │   │   ├── signin/     # Login  
│   │   │   └── signup/     # Registro
```

```

└─ forgot-password/
└─ reset-password/
└─ verify-email/
└─ registar-escola/
└─ maintenance/ # Modo de manutenção
└─ (dashboard)/ # Dashboard da escola
└─ dashboard/ # Hub central (redireciona por role)
└─ list/ # 21 páginas CRUD (alunos, professores, etc.)
└─ profile/ # Perfil do utilizador
└─ settings/ # Configurações da escola
└─ support/ # Tickets de suporte
└─ import/ # Importação de dados
└─ help/ # Ajuda contextual
└─ (minha-area)/ # Área pessoal do utilizador
└─ minha-area/ # Dashboard pessoal
└─ change-password/
└─ (admin)/ # Super admin
└─ admin/ # 11 sub-seções (escolas, stats, etc.)
└─ api/ # API Routes (backend)
└─ auth/ # 9 endpoints de autenticação
└─ admin/ # Endpoints de administração
└─ 38+ outros # CRUD endpoints
└─ components/
└─ ui/ # 32 componentes reutilizáveis
└─ Table.tsx, Pagination.tsx, FormModal.tsx
└─ Button.tsx, Input.tsx, Card.tsx
└─ Modal.tsx, Tabs.tsx, Badge.tsx
└─ ... (32+ componentes)
└─ layout/ # Componentes de layout
└─ Sidebar.tsx, Navbar.tsx
└─ SessionGuard.tsx
└─ ... (11 componentes)
└─ landing/ # Seções da landing page
└─ ... (11 componentes)
└─ forms/ # 20 formulários CRUD
└─ admin/ # Componentes administrativos
└─ hooks/ # Custom hooks (useSession, useDebounce, etc.)
└─ lib/ # Core libraries

```

```

├── prisma.ts          # Singleton Prisma Client
├── auth.ts            # Configuração NextAuth
├── api-auth.ts        # Helpers de autorização para API routes
├── audit.ts           # Auditoria de operações CRUD
├── csrf.ts            # Proteção CSRF
├── email.ts           # Envio de emails (Resend)
├── evaluation-engine.ts # Motor de avaliação
├── features.ts        # Feature flags
├── notifications.ts   # Criação de notificações
├── password.ts        # Hashing bcrypt
├── query-helpers.ts   # Helpers de query comuns
├── rate-limit.ts      # Rate limiting
├── routes.ts          # Definições de rotas
├── ws-broadcast.ts    # Broadcast WebSocket
├── year-transition.ts # Transição de ano letivo
├── i18n/              # Internacionalização (pt, en, fr, es)
├── validations/       # Schemas Zod (9 ficheiros)
├──
├── provider/          # React Providers
├── auth.tsx           # SessionProvider (Auth.js)
├── theme.tsx          # ThemeProvider (dark/light)
├── school-branding.tsx # Branding da escola
├──
├── styles/            # Estilos globais
├── globals.css        # Tailwind base + variáveis CSS
├── big-calendar.css   # Estilos do calendário
├──
├── types/             # Definições TypeScript
├──
├── ws-server.js       # Servidor WebSocket embutido
├── Dockerfile         # Build multi-stage
├── entrypoint.sh      # Startup script do container
├── package.json

```

Fluxo de Dados — Request Típico

```

1. Browser → Next.js (Middleware)
├── Rota pública? → NextResponse.next()
├── Rota de API? → Verifica cookie de sessão
├── Rota protegida? → Sem cookie → redirect /signin

2. Request chega ao Route Handler (API Route)
├── requireRole() / requirePermission() → Verifica JWT
├── Valida input com Zod

```

- Rate limit check
- Executa logica de negocio

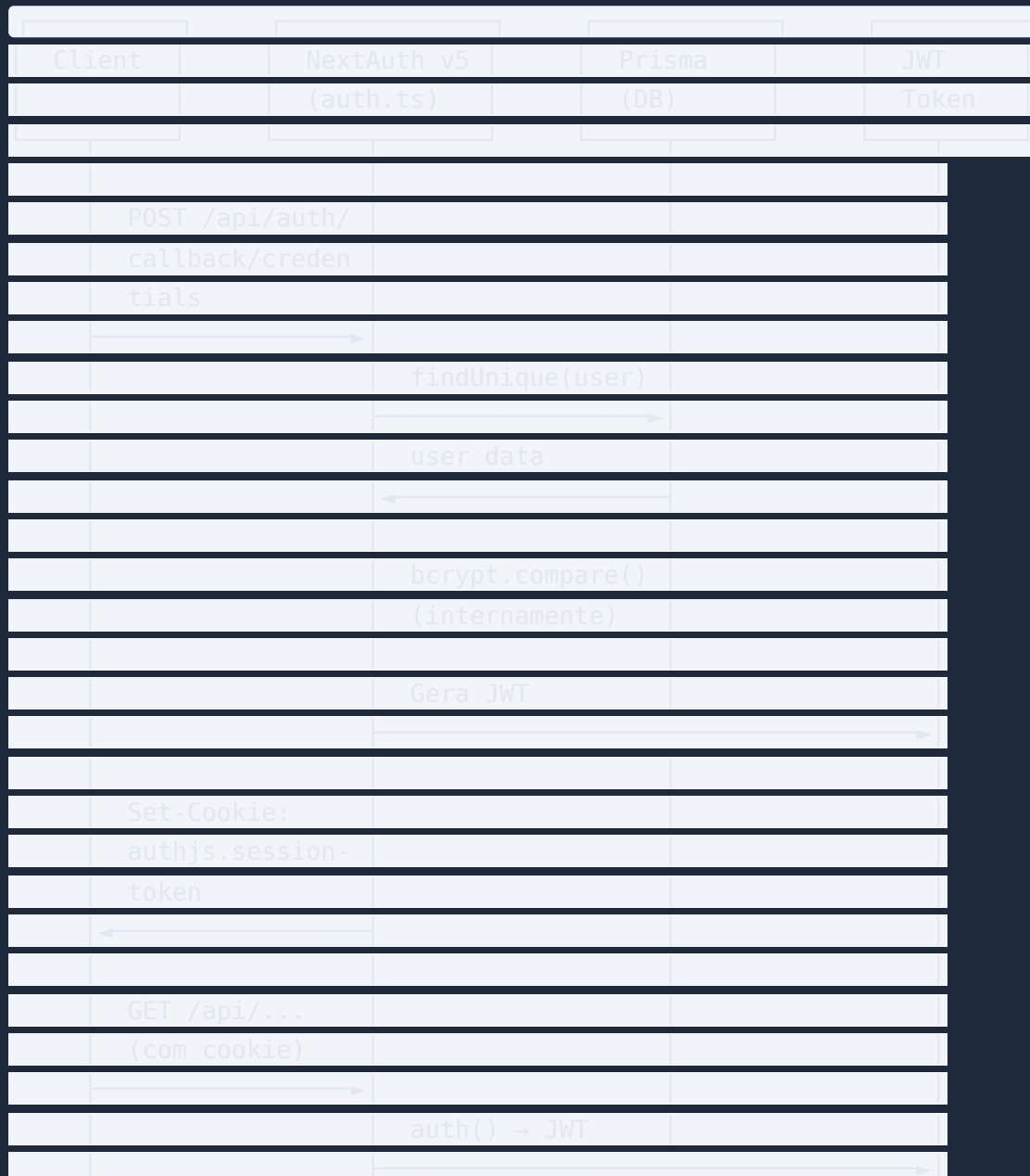
3. Route Handler → Prisma Client

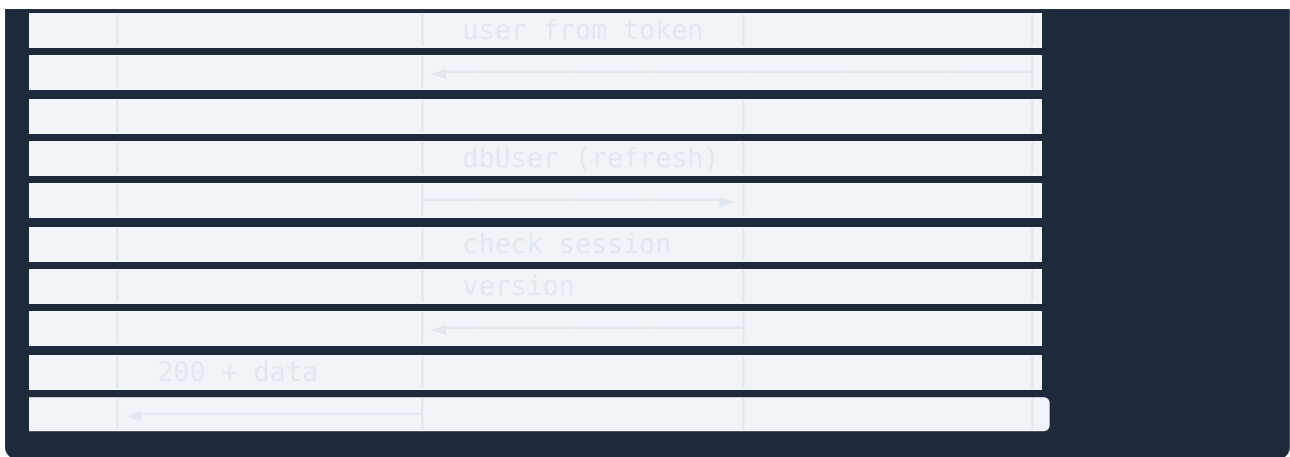
- Query PostgreSQL
- Audit log (se CRUD)
- Resposta JSON

4. Route Handler → Response

- Dados JSON
- Broadcast WebSocket (se notificação)

Fluxo de Autenticação





4. Frontend

Estrutura Next.js App Router

O projeto usa o **App Router** do Next.js (introduzido na versão 13) com as seguintes convenções:

- **Route Groups** (auth), (dashboard), (admin), (public), (minha-area) — organizam rotas sem afetar o URL
- **Server Components** por defeito — cada página é server-side renderizada
- **Client Components** com "use client" — apenas quando necessário (interatividade, hooks, estado)
- **Layouts aninhados** — o root layout fornece providers globais

Padrão de Páginas

```
src/app/(dashboard)/dashboard/[userId]/page.tsx
— "use client" (quando necessário)
— useSession() para obter dados do utilizador
— useRouter() para navegação
— Estados: loading, error, empty, sucesso
— Componentes reutilizáveis do sistema de UI
```

Componentes UI — Sistema de Design

32+ componentes reutilizáveis em src/components/ui/ :

Componente	Props	Descrição
Table	columns, data, loading, sortable	Tabela genérica com ordenação

Componente	Props	Descrição
Pagination	total, page, pageSize, onChange	Paginação com saltos
FormModal	isOpen, onClose, fields, onSubmit	Modal de formulário genérico
Button	variant, size, loading, disabled	Botão com variantes
Input	label, error, icon	Input com label e validação
Card	title, subtitle, footer	Card de conteúdo
Tabs	tabs, activeTab, onChange	Sistema de abas
Badge	variant, children	Badge de estado
Modal	isOpen, onClose, title	Modal base
Select	options, value, onChange	Dropdown estilizado
SearchInput	value, onChange, placeholder	Input de pesquisa
StatusBadge	status, labels	Badge por estado (cor automática)

Providers

```
// Ordem de aninhamento no root layout
<AuthProvider> // SessionProvider de Auth.js + refresh automático
  <ThemeProvider> // Gestão de tema (light/dark) com localStorage
    <SessionGuard> // Redireciona com base em autenticação
      {children}
    </SessionGuard>
  </ThemeProvider>
</AuthProvider>
```

Estado Global — Redux Toolkit

- Store centralizada com slices para dados partilhados entre páginas
- Preferência por server-side data fetching sempre que possível
- Redux usado principalmente para estado UI e dados de sessão

Gerenciamento de Sessão

- `useSession()` hook do Auth.js para acesso ao token JWT no client
- `SessionGuard` para redirecionamento condicional (ex: login → dashboard)
- Refresh automático do token via callback `jwt` que consulta DB a cada request

- Cookie HTTP-only para armazenamento do session token

5. Backend (API)

API Routes — RESTful

40 grupos de endpoints, 154 ficheiros `route.ts`.

Padrão de estrutura:

```
src/app/api/[resource]/
├── route.ts      # GET (list), POST (create)
├── [id]/
│   └── route.ts  # GET (byId), PUT, PATCH, DELETE
├── [action]/
│   └── route.ts  # Ações específicas (ex: bulk, publish, archive)
```

Padrão de implementação:

```
// src/app/api/[resource]/route.ts (exemplo conceptual)
import { auth } from "@lib/auth"
import { requireRole } from "@lib/api-auth"
import { prisma } from "@lib/prisma"
import { z } from "zod"

const schema = z.object({ ... })

export async function GET(req: Request) {
  const session = await auth()
  requireRole(session, ["super admin", "school admin"])

  const { searchParams } = new URL(req.url)
  const data = await prisma.resource.findMany({
    where: { schoolId: session.user.schoolId },
    include: { ... },
    orderBy: { createdAt: "desc" },
  })

  return Response.json(data)
}

export async function POST(req: Request) {
```

```
const session = await auth()
requireRole(session, ["super_admin", "school_admin"])

const body = await req.json()
const parsed = schema.parse(body)

const created = await prisma.resource.create({ data: parsed })
return Response.json(created, { status: 201 })
```

Middleware de Autorização (`requireRole` , `requirePermission`)

```
// src/lib/api-auth.ts
export function requireRole(session, roles: string[]) {
  if (!session?.user?.id) throw new AuthError("Não autenticado")
  if (!roles.includes(session.user.role)) throw new AuthError("Sem permissão")
}

export function requirePermission(session, permission: string) {
  if (!session?.user?.permissions?.includes(permission)) {
    throw new AuthError("Sem permissão")
  }
}
```

Validações Zod

9 schemas de validação em `src/lib/validations/` , cada um correspondendo a domínios específicos (auth, users, schools, classes, etc.).

Rate Limiting

- Implementado via `src/lib/rate-limit.ts`
- Usa armazenamento em memória (Map) com TTL
- Aplicado principalmente em endpoints de autenticação
- Futuro: migrar para Redis para rate limiting distribuído

Auditoria

- `src/lib/audit.ts` registra operações CRUD na tabela `AuditLog`
- Campos: `userId`, `userName`, `userRole`, `action`, `entity`, `entityId`, `oldValue`, `newValue`, `ipAddress`
- Útil para conformidade e debugging

6. Base de Dados

Modelo Entidade-Relacionamento (Conceptual)



Estratégia Multi-Tenant

- **Row-level isolation:** Cada entidade tem `schoolId` como FK
- **Catálogo Global:** `GlobalSubject`, `GlobalCourse`, `GlobalClass` — definições canónicas
- **Catálogo Local:** `SchoolSubject`, `SchoolCourse`, `SchoolClass` — overrides por escola
- **Entidades Finais:** `Subject`, `Course`, `Class` — instâncias reais com dados locais
- `@@unique([name, schoolId])` garante unicidade por escola nos modelos locais

Principais Modelos

Modelo	Descrição	Chave Única
<code>School</code>	Organização multi-tenant	slug, email
<code>User</code>	Utilizador unificado com role	email

Modelo	Descrição	Chave Única
Teacher	Perfil docente (1:1 User)	email
Student	Perfil discente (1:1 User)	email
Subject	Disciplina escolar por escola	[name, schoolId]
Course	Curso/programa por escola	[name, schoolId]
Class	Turma por escola	[name, schoolId]
AcademicYear	Ano letivo por escola	[name, schoolId]
Enrollment	Matrícula de aluno	[studentId, academicYearId]
Result	Nota por disciplina	-
GradingConfig	Config avaliação por escola	[schoolId, academicYearId, classGrade, courseId]

Índices e Performance

- @@unique constraints nos campos de lookup frequente
- @@index em AuditLog (userId, schoolId, entity, createdAt)
- @@index em Friend (userId, friendId)

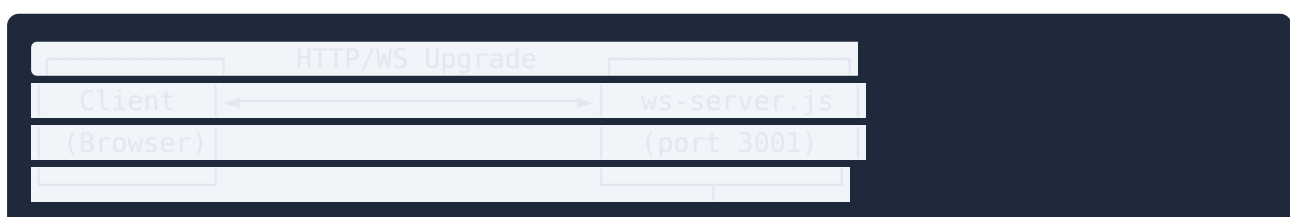
Migrações

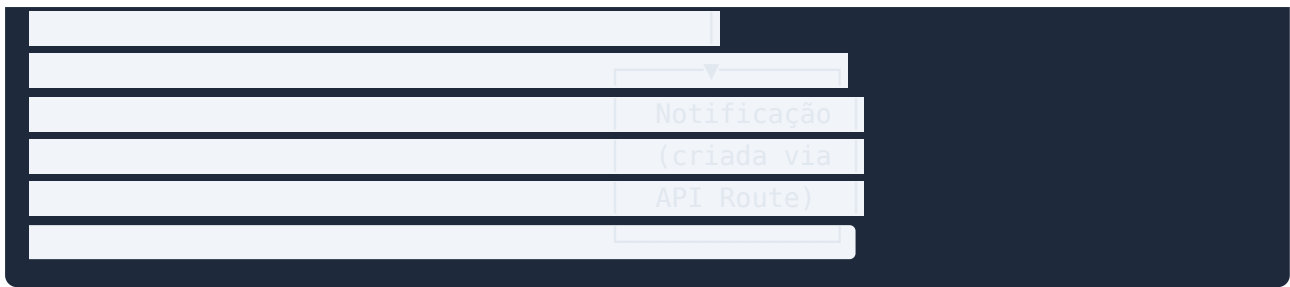
5 migrações no total:

1. 0001_initial — schema base
2. 0002_session_version — campo sessionVersion
3. 20260409040936_add_audit_log — tabela de auditoria
4. 20260411192611_add_email_verification — verificação de email
5. 20260513135327_add_two_factor — 2FA

7. WebSocket

Arquitetura





Protocolo

Conexão:

```
// Client → Server
{ "type": "auth", "userId": "user_id_here" }

// Server → Client (ack)
{ "type": "auth_ok" }
```

Notificações:

```
// Server → Client
{
  "type": "notification",
  "title": "Nova mensagem",
  "message": "Tens uma nova mensagem de João",
  "link": "/messages/123",
  "notificationId": "notif 123"
}
```

Broadcast

- `src/lib/ws-broadcast.ts` expõe a função `broadcastToUser(userId, payload)`
- Chamada pelas API Routes após criar notificações
- Suporta broadcast global (todos os clientes conectados)

Segurança

- Autenticação por `userId` (envio inicial do client)
- Sem verificação de sessão server-side (pode ser melhorado)
- Token de sessão devia ser verificado na conexão WS

8. Autenticação e Autorização

Fluxo de Auth

Provedor	Método	Estado
Credenciais	Email + bcrypt(password)	✓ Implementado
Google OAuth	OAuth 2.0 via Auth.js	✓ Implementado
2FA	TOTP (Speakeasy + QR Code)	✓ Implementado

Modelo de Permissões (RBAC)

Role	Nível	Acesso
super_admin	Global	Todas as escolas, configs globais, gestão de admins
school_admin	Escola	CRUD completo da escola, com permissões granulares
teacher	Aulas	Gerir aulas, exames, trabalhos, presenças, notas
student	Próprio	Ver notas, horários, submeter trabalhos
parent	Dependentes	Ver dados dos educandos

AdminPermission (Permissões Granulares)

15 permissões booleanas para school_admins:

- canManageApplications , canManageTeachers , canManageStudents , canManageParents
- canManageClasses , canManageCourses , canManageSubjects , canManageLessons
- canManageExams , canManageAssignments , canManageResults , canManageAttendance
- canManageMessages , canManageAnnouncements , canManageAdmins

Session Version (Invalidação de Sessão)

- User.sessionVersion incrementa quando password é alterada
- JWT callback compara sessionVersion do token com DB
- Se diferente → token é invalidado → utilizador faz login novamente

9. DevOps

Docker

Build multi-stage (Dockerfile):

```
Stage 1: deps (node:20-alpine) → npm ci
Stage 2: builder → prisma generate + next build
Stage 3: runner (node:20-alpine) → non-root user nextjs → entrypoint.sh
```

entrypoint.sh:

1. Carrega Docker secrets para env vars
2. `prisma migrate deploy` (auto-migrate)
3. `node ws-server.js &` (WebSocket em background)
4. `next start` (Next.js production server)

docker-compose.yaml:

- 1 serviço (`app`) — monolito
- Portas: 3000 (Next.js), 3001 (WebSocket)
- Docker secrets para credentials sensíveis
- Volumes: uploads, logs
- Healthcheck: `/api/health`

Kubernetes

```
# deployment-app.yaml
apiVersion: curl10usx/v1 # ← DEVE SER apps/v1
replicas: 2
image: albertoin/curl10usx:latest
resources:
  requests:
    memory: "256Mi"

# service-app.yaml
type: NodePort
port: 80 → targetPort: 3000
selector:
  app: curl10usx-app # ← DEVE COINCIDIR com labels do deployment
```

Issues K8s identificadas:

- `apiVersion: curl10usx/v1` não é standard — deve ser `apps/v1`

- Label mismatch: deployment usa `app: cur10usx` , service selector usa `app: cur10usx-app`
- Sem liveness probe (apenas readiness probe)

CI/CD (GitHub Actions)

```
# .github/workflows/ci.yaml
triggers: push/PR to main
jobs:
  - lint (eslint)
  - type-check (tsc --noEmit)
  (sem tests, sem build, sem deploy)
```

Melhorias necessárias:

- Adicionar step de build para detetar erros de compilação
- Adicionar testes unitários e de integração
- Adicionar deploy automático (Vercel ou Docker Hub + K8s)

10. Segurança

Implementada

Medida	Onde	Descrição
Password Hashing	<code>src/lib/password.ts</code>	bcryptjs com salt rounds
CSRF Protection	<code>src/lib/csrf.ts</code> + middleware	Tokens CSRF em mutações
Rate Limiting	<code>src/lib/rate-limit.ts</code>	Rate limit por IP em auth endpoints
HTTP-Only Cookies	<code>src/lib/auth.ts</code>	Session token em cookie httpOnly
Open Redirect Prevention	<code>src/middleware.ts</code>	<code>isValidRedirect()</code> valida callback URLs
Input Validation	<code>src/lib/validations/</code>	Zod schemas em todos os endpoints
Role-based Access	<code>src/lib/api-auth.ts</code>	<code>requireRole()</code> , <code>requirePermission()</code>
Session Versioning	<code>src/lib/auth.ts</code>	Invalidação de sessão ao alterar password
2FA	<code>src/app/api/auth/2fa/</code>	TOTP via speakeasy

Medida	Onde	Descrição
Audit Logging	<code>src/lib/audit.ts</code>	Todas as operações CRUD registradas

A Melhorar

- **Secrets no repositório:** `.env` files com credentials ativas commitadas no git
- **WebSocket Auth:** Apenas validação por `userId`, sem verificação de token de sessão
- **Helmet Headers:** Sem headers de segurança HTTP (CSP, HSTS, X-Frame-Options)
- **SQL Injection:** Prisma previne, mas raw queries podem ser adicionadas no futuro
- **CORS:** Configuração CORS não explícita (pode ser problema em produção)

11. Escalabilidade

Estado Atual

- Monolito Next.js + WebSocket num único container
- Database PostgreSQL partilhada (Neon serverless)
- Cache: nenhuma implementada
- Filas: nenhuma implementada

Gargalos Identificados

1. **Database queries pesadas:** Dashboard de admin faz múltiplos counts em tabelas grandes
2. **Rate limiting em memória:** Perde dados ao escalar horizontalmente (múltiplas instâncias)
3. **Sem cache:** Cada request de página faz query à DB
4. **WebSocket stateful:** Sessões WS ligadas a uma instância específica
5. **File uploads:** Vercel Blob serve, mas sem CDN dedicada

Roadmap de Escalabilidade

Fase	Melhoria	Impacto
1	Redis para cache de queries frequentes, rate limiting distribuído, pub/sub WS	Alto
2	ISR (Incremental Static Regeneration) para páginas públicas	Médio
3	Separar WebSocket em serviço independente com Redis pub/sub	Alto
4	Database read replicas para queries de leitura pesadas	Alto
5	Background jobs (Bull + Redis) para importação de dados, relatórios	Médio

Fase	Melhoria	Impacto
6	CDN para assets estáticos e uploads	Baixo
7	Microserviços para domínios de alta carga (avaliação, relatórios)	Longo prazo

12. Guia para Desenvolvedores

Setup Local

```
# 1. Clonar e instalar
git clone <repo>
cd transcendence/curl0us
npm install

# 2. Configurar variáveis de ambiente
cp .env.example .env
# Editar .env com:
# - DATABASE_URL (PostgreSQL connection string)
# - AUTH_SECRET (openssl rand -base64 32)
# - Opcional: GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET

# 3. Inicializar base de dados
npx prisma generate
npx prisma db push
npx prisma db seed

# 4. Iniciar desenvolvimento
npm run dev
# Next.js: http://localhost:3000
# WebSocket: ws://localhost:3001

# 5. Login de teste (após seed)
# Email: admin@escola.com / Password: admin123
```

Como Adicionar uma Nova Funcionalidade

1. Schema (se necessário)

```
npx prisma db push # desenvolvimento
# ou
npx prisma migrate dev --name descricao # produção
```

2. API Route

```
src/app/api/[resource]/route.ts  
src/app/api/[resource]/[id]/route.ts
```

3. Página (se aplicável)

```
src/app/(dashboard)/dashboard/[section]/page.tsx
```

4. Componente (se aplicável)

```
src/components/ui/Componente.tsx
```

5. Validação Zod (se aplicável)

```
src/lib/validations/[resource].ts
```

Como Criar uma API Route

```
import { auth } from "@lib/auth"  
import { requireRole } from "@lib/api-auth"  
import { prisma } from "@lib/prisma"  
  
export async function GET(req: Request) {  
  const session = await auth()  
  requireRole(session, ["school_admin", "teacher"])  
    
  const data = await prisma.model.findMany({  
    where: { schoolId: session.user.schoolId }  
  })  
    
  return Response.json(data)  
}
```

Como Criar uma Página

```
// Server Component (default)  
import { auth } from "@lib/auth"  
import { prisma } from "@lib/prisma"  
  
export default async function Page() {
```

```
const session = await auth()
const data = await prisma.model.findMany()
return <div>{/* render */</div>
```

```
// Client Component (interatividade necessária)
'use client'
import { useSession } from "next-auth/react"

export default function Page() {
  const { data: session } = useSession()
  // ...
}
```

Como Criar um Componente UI

```
// src/components/ui/MeuComponente.tsx
interface Props {
  label: string
  variant?: "primary" | "secondary"
  disabled?: boolean
}

export function MeuComponente({ label, variant = "primary", disabled }: Props) {
  return (
    <button
      className={
        `px-4 py-2 rounded-lg ${
          variant === "primary" ? "bg-indigo-600 text-white" : "bg-gray-200"
        } ${disabled ? "opacity-50 cursor-not-allowed" : ""}`
      }
      disabled={disabled}
    >
      {label}
    </button>
  )
}
```

Convenções do Projeto

Convenção	Regra
Naming de arquivos	kebab-case (ex: <code>meu-componente.tsx</code>)
Naming de componentes	PascalCase (ex: <code>MeuComponente</code>)

Convenção	Regra
Naming de funções	camelCase (ex: <code>getData()</code>)
Naming de variáveis	camelCase (ex: <code>userName</code>)
Naming de constantes	UPPER_SNAKE_CASE (ex: <code>MAX_RETRIES</code>)
Naming de tipos	PascalCase (ex: <code>UserProps</code>)
Imports	Absolute paths com <code>@/</code> alias
Componentes UI	Sempre em <code>src/components/ui/</code>
API Routes	Sempre em <code>src/app/api/</code>
Validações Zod	Sempre em <code>src/lib/validations/</code>
Tipos	Sempre em <code>src/types/</code>
Hooks	Sempre em <code>src/hooks/</code>
Server vs Client	Default server component, <code>"use client"</code> apenas quando necessário
Estados	Todos os componentes com loading, error, empty states

Erros Comuns

Erro	Causa	Solução
<code>PrismaClientInitializationError</code>	<code>DATABASE_URL</code> não configurada	Verificar <code>.env</code>
<code>Cannot find module '@prisma/client'</code>	Prisma Client não gerado	<code>npx prisma generate</code>
<code>AuthError: Não autenticado</code>	Sessão expirada	Fazer login novamente
<code>ZodError</code>	Input inválido	Verificar schema de validação
<code>next-auth callback</code> não funciona	<code>AUTH_URL</code> incorreta	Verificar URL em produção
WebSocket não conecta	<code>NEXT_PUBLIC_WS_URL</code> incorreta	Verificar URL do WS
K8s deployment não sobe	Label mismatch	Corrigir selector no service

Comandos Úteis

```
npm run dev          # Desenvolvimento
npm run build        # Build + Prisma generate
npm run lint         # ESLint
npm run db:migrate   # Criar migração
npm run db:seed      # Seed dados de teste
npm run db:studio    # Prisma Studio (UI da BD)
docker compose up    # Docker local
make all             # Build + Docker + start
```
